UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

DEPARTMENT OF INFORMATION COMMUNICATION AND TECHNOLOGY

--------***--------

REPORT LABWORK 4

MACHINE LEARNING AND DATA MINING II

CLASS: 1

	STUDENT NAME	STUDENT ID
1	Nguyễn Quang Hưng	22BI13184
2	Vũ Tuấn Hải	22BI13149

Ha Noi, March 18th 2025

For this labwork, we select a total of 4 datasets available in the *scikit-learn* library. In the first part, the two datasets are Iris and Breast Cancer. In the second part, we select two datasets, namely the Wine dataset and Digits dataset.

Decision Tree - DT

I. Datasets

1. Dataset analysis

We examined key properties such as:

a) Iris dataset

- **Feature Names:** The four measured attributes (sepal length, sepal width, petal length, petal width).
- **Target Names:** The three species of iris.
- **Data Shape:** Confirming that there are 150 samples.

b) Breast Cancer dataset

- **Feature Names:** A list of 30 numeric attributes describing cell characteristics.
- **Target Classes:** The dataset contains two classes: malignant and benign.
- **Data Shape:** Verification that there are 569 samples.

2. Preprocessing

- Minimal preprocessing was required for the Iris dataset due to its good curation and absence of missing values.
- Similar to that, the Breast Cancer dataset is pre-cleaned. No major data cleaning or imputation was required with no application of feature scaling.

II. Divide datasets into two subsets

For both datasets, we set an 80-to-20 ratio:

- 80% Training Set: used to build the model
- 20% Testing Set: used to evaluate the model

```
X_train_iris, X_test_iris, y_train_iris, y_test_iris = train_test_split(
    X_iris, y_iris, test_size=0.2, random_state=42)

X_train_cancer, X_test_cancer, y_train_cancer, y_test_cancer = train_test_split(
    X_cancer, y_cancer, test_size=0.2, random_state=42)
```

III. Build Decision Tree DT

We used the `DecisionTreeClassifier` from *sklearn.tree* to construct the DT model. The model was trained using the training subset of each dataset.

```
dt_iris = DecisionTreeClassifier(random_state=42)
dt_iris.fit(X_train_iris, y_train_iris)
```

IV. Calculate the error of classification

After training, the DT model was used to predict the classes for the testing set.
The classification error was calculated as:

$$Error = 1 - Accuracy$$

We use function from *sklearn.metrics* to compute accuracy

```
Accuracy: 100.00%
Classification Error: 0.00%
```

Random Forest - RF

I. Datasets

1. Data analysis

We examined key properties such as:

a) Wine dataset

- **Source:** The Wine dataset is available from the UCI Machine Learning Repository via scikit-learn.
- **Features:** This dataset contains 13 chemical analysis measurements (e.g., alcohol, malic acid, color intensity) of wines.
- **Classes:** There are three wine cultivars (classes).
- **Size:** The dataset includes 178 samples.

b) Digits dataset

- **Source:** The Digits dataset is also available via scikit-learn.
- **Features:** Each sample is an 8×8 image of a handwritten digit, flattened into 64 features.
- **Classes:** There are 10 classes corresponding to the digits 0 through 9.
- **Size:** The dataset comprises 1,797 samples.

2. Preprocessing

Exploratory Analysis:

- For Wine, the feature names and class labels were reviewed to understand the chemical attributes.
- For Digits, the structure of the flattened image data was examined, and unique class labels were identified.

Pre-processing:
Both datasets are well-curated. As they are numerical and do not contain missing values, no extensive pre-processing (e.g., imputation or scaling) was required for the models employed. The inherent properties of the data were maintained to focus on model performance.

II. Build Testing set

Using bootstrapping, 100 distinct training sets were created from the original training data for each dataset. This was accomplished by randomly sampling (with replacement) from the training set.

```
K = 100
rf_errors_wine = []
```

III. Build Random Forest RF

For each of the 100 bootstrapped training sets, a Random Forest classifier (with 100 trees per forest) was built using `RandomForestClassifier` from *scikit-learn*.

```
for i in range(K):
    X_train_boot, y_train_boot = resample(
        X_train_wine, y_train_wine, replace=True, n_samples=len(y_train_wine), random_state=i)

  rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
  rf_model.fit(X_train_boot, y_train_boot)
```